

Krishnamargam - Final Database Schema

Version: 2.0
Date: May 2026
Database: PostgreSQL (via Supabase)
Maintained by: BabyStepsIndia.in

1. Overview

Group	Tables	Purpose
Platform	paths	Learning path definitions (RAAS, DAAS)
Content	courses, lessons, quiz_questions	Course, lesson, and quiz content
Users	profiles	Extended user data
Enrollment	enrollments, enrollment_intents	Student access + intent tracking
Learning	user_progress, watch_sessions, user_notes	Progress and watch tracking
Quiz	quiz_attempts	Student quiz results
Q&A	qa_questions, qa_replies	Per-lesson Q&A
Transactions	payment_logs	Razorpay payment records
Community	testimonials	Student reviews

2. Full Schema (SQL)

```
sql
```

```

-- =====
-- EXTENSIONS
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";


-- =====
-- GROUP 1: PLATFORM
-- =====

CREATE TABLE paths (
  id          SERIAL PRIMARY KEY,
  slug        TEXT UNIQUE NOT NULL,
  name        TEXT NOT NULL,
  emoji       TEXT NOT NULL DEFAULT '🏠',
  tagline_en  TEXT NOT NULL,
  tagline_te  TEXT,
  description_en TEXT,
  description_te TEXT,
  is_active   BOOLEAN NOT NULL DEFAULT FALSE,
  order_index INTEGER NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE paths IS 'Spiritual learning paths (RAAS, DAAS, etc.)';
COMMENT ON COLUMN paths.is_active IS 'Only active paths are visible to students';
COMMENT ON COLUMN paths.order_index IS 'Controls display order on home page';


-- =====
-- GROUP 2: CONTENT
-- =====

CREATE TABLE courses (
  id          SERIAL PRIMARY KEY,
  path_id     INTEGER NOT NULL REFERENCES paths(id) ON DELETE RESTRICT,
  slug        TEXT UNIQUE NOT NULL,
  emoji       TEXT NOT NULL DEFAULT '📖',
  bg_color    TEXT NOT NULL DEFAULT '#1a0a00',
  title_en    TEXT NOT NULL,
  title_te    TEXT,

```

```

description_en TEXT,
description_te TEXT,
instructor_en TEXT NOT NULL,
instructor_te TEXT,
category TEXT NOT NULL,
level TEXT NOT NULL CHECK (level IN ('Beginner','Intermediate','Advanced')),
badge TEXT NOT NULL CHECK (badge IN ('Popular','New','Free')),
duration TEXT,
is_free BOOLEAN NOT NULL DEFAULT FALSE,
price INTEGER NOT NULL DEFAULT 100, -- INR, default ₹100
rating NUMERIC(2,1) DEFAULT 0.0 CHECK (rating >= 0 AND rating <= 5),
review_count INTEGER NOT NULL DEFAULT 0,
student_count INTEGER NOT NULL DEFAULT 0,
has_quiz BOOLEAN NOT NULL DEFAULT FALSE, -- whether course has a completion quiz
quiz_pass_pct INTEGER NOT NULL DEFAULT 80, -- minimum % to pass quiz (default 80%)
order_index INTEGER NOT NULL DEFAULT 0,
is_published BOOLEAN NOT NULL DEFAULT FALSE,
is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

COMMENT ON TABLE courses IS 'Admin-defined courses. No student or open submission of course content';
COMMENT ON COLUMN courses.price IS 'Price in INR. Default ₹100. Admin can override price for a course';
COMMENT ON COLUMN courses.has_quiz IS 'If TRUE, student must pass quiz to mark course as completed';
COMMENT ON COLUMN courses.quiz_pass_pct IS 'Minimum score percentage required to pass the course quiz';
COMMENT ON COLUMN courses.is_deleted IS 'Soft delete – data preserved for enrolled students';

```

```

CREATE INDEX idx_courses_path_id ON courses(path_id);
CREATE INDEX idx_courses_published ON courses(is_published, is_deleted);
CREATE INDEX idx_courses_order ON courses(path_id, order_index);

```

```

CREATE TABLE lessons (
  id SERIAL PRIMARY KEY,
  course_id INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  section_title TEXT,
  title_en TEXT NOT NULL,
  title_te TEXT,
  youtube_video_id TEXT NOT NULL, -- stored server-side only, never sent to client
  duration TEXT,
  order_index INTEGER NOT NULL DEFAULT 0,
  is_preview BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```

```

    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE lessons IS 'Video lessons within a course';
COMMENT ON COLUMN lessons.youtube_video_id IS 'YouTube video ID. Never exposed to client';
COMMENT ON COLUMN lessons.is_preview IS 'If TRUE, lesson is watchable without enrollment';

CREATE INDEX idx_lessons_course_id ON lessons(course_id);
CREATE INDEX idx_lessons_order ON lessons(course_id, order_index);


CREATE TABLE quiz_questions (
    id                  SERIAL PRIMARY KEY,
    course_id           INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
    question_text       TEXT NOT NULL,
    option_a            TEXT NOT NULL,
    option_b            TEXT NOT NULL,
    option_c            TEXT NOT NULL,
    option_d            TEXT NOT NULL,
    correct_option      CHAR(1) NOT NULL CHECK (correct_option IN ('a','b','c','d')),
    order_index         INTEGER NOT NULL DEFAULT 0,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE quiz_questions IS 'MCQ questions for course completion quiz';
COMMENT ON COLUMN quiz_questions.correct_option IS 'The letter of the correct answer';

CREATE INDEX idx_quiz_questions_course ON quiz_questions(course_id);


-- =====
-- GROUP 3: USERS
-- =====


CREATE TABLE profiles (
    id                  UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    full_name           TEXT NOT NULL,
    mobile              TEXT,
    isd_code            TEXT NOT NULL DEFAULT '+91',
    mobile_verified     BOOLEAN NOT NULL DEFAULT FALSE, -- verified via WhatsApp OTP
    avatar_initials     TEXT,
    is_admin            BOOLEAN NOT NULL DEFAULT FALSE,
    preferred_lang      TEXT NOT NULL DEFAULT 'en' CHECK (preferred_lang IN ('en','te')),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```

```

    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE profiles IS 'Extended user data linked to Supabase auth.users';
COMMENT ON COLUMN profiles.mobile_verified IS 'TRUE after WhatsApp OTP verification';
COMMENT ON COLUMN profiles.is_admin IS 'Admin users can access /admin/* routes';
COMMENT ON COLUMN profiles.avatar_initials IS 'Computed at registration from first 1

-- =====
-- GROUP 4: ENROLLMENT
-- =====

CREATE TABLE enrollments (
    id              SERIAL PRIMARY KEY,
    user_id         UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    course_id       INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
    enrolled_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    is_active       BOOLEAN NOT NULL DEFAULT FALSE,
    activated_at    TIMESTAMPTZ,
    UNIQUE (user_id, course_id)
);

COMMENT ON TABLE enrollments IS 'Student course access. is_active=FALSE until payment';
COMMENT ON COLUMN enrollments.is_active IS 'Set TRUE immediately for options 2 & 3,

CREATE INDEX idx_enrollments_user ON enrollments(user_id);
CREATE INDEX idx_enrollments_course ON enrollments(course_id);
CREATE INDEX idx_enrollments_active ON enrollments(user_id, is_active);

CREATE TABLE enrollment_intents (
    id              SERIAL PRIMARY KEY,
    user_id         UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    course_id       INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
    intent          TEXT NOT NULL CHECK (intent IN ('paid', 'cannot_pay', 'not_willing_to_
    amount_paid     INTEGER NOT NULL DEFAULT 0,    -- 100 for paid, 0 for others
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (user_id, course_id)
);

COMMENT ON TABLE enrollment_intents IS 'Records which of the 3 enrollment options a
COMMENT ON COLUMN enrollment_intents.intent IS 'paid=Option1(₹100), cannot_pay=Optio
COMMENT ON COLUMN enrollment_intents.amount_paid IS '100 for paid intent, 0 for opti

```

```

CREATE INDEX idx_intents_user ON enrollment_intents(user_id);
CREATE INDEX idx_intents_course ON enrollment_intents(course_id);
CREATE INDEX idx_intents_intent ON enrollment_intents(intent);
CREATE INDEX idx_intents_created ON enrollment_intents(created_at);

-- =====
-- GROUP 5: LEARNING
-- =====

CREATE TABLE user_progress (
  id SERIAL PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  course_id INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  lesson_id INTEGER NOT NULL REFERENCES lessons(id) ON DELETE CASCADE,
  completed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, lesson_id)
);

COMMENT ON TABLE user_progress IS 'Records which lessons each student has completed'

CREATE INDEX idx_progress_user_course ON user_progress(user_id, course_id);

CREATE TABLE watch_sessions (
  id SERIAL PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  lesson_id INTEGER NOT NULL REFERENCES lessons(id) ON DELETE CASCADE,
  course_id INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  max_watch_pct INTEGER NOT NULL DEFAULT 0 CHECK (max_watch_pct >= 0 AND max_w
  total_watch_seconds INTEGER NOT NULL DEFAULT 0,
  session_date DATE NOT NULL DEFAULT CURRENT_DATE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE watch_sessions IS 'YouTube iFrame API watch-time tracking per lesson'
COMMENT ON COLUMN watch_sessions.max_watch_pct IS 'Highest % of the video reached in session'
COMMENT ON COLUMN watch_sessions.total_watch_seconds IS 'Cumulative seconds watched'

CREATE INDEX idx_watch_user_lesson ON watch_sessions(user_id, lesson_id);
CREATE INDEX idx_watch_session_date ON watch_sessions(session_date);
CREATE INDEX idx_watch_course ON watch_sessions(course_id);

```

```

CREATE TABLE user_notes (
  id          SERIAL PRIMARY KEY,
  user_id     UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  course_id   INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  content     TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, course_id)
);

COMMENT ON TABLE user_notes IS 'Private freeform notes per student per course. Not v

CREATE INDEX idx_notes_user_course ON user_notes(user_id, course_id);

-- =====
-- GROUP 6: QUIZ
-- =====

CREATE TABLE quiz_attempts (
  id          SERIAL PRIMARY KEY,
  user_id     UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  course_id   INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  answers     JSONB NOT NULL,                -- {"question_id": "a", ...}
  total_questions INTEGER NOT NULL,
  correct_answers INTEGER NOT NULL,
  score_pct   INTEGER NOT NULL,              -- (correct/total)*100
  passed      BOOLEAN NOT NULL,              -- score_pct >= courses.quiz_pass_pct
  attempted_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE quiz_attempts IS 'Each quiz attempt by a student. Multiple attempts
COMMENT ON COLUMN quiz_attempts.answers IS 'JSONB map of question_id to chosen optio
COMMENT ON COLUMN quiz_attempts.passed IS 'TRUE if score_pct >= course quiz_pass_pct

CREATE INDEX idx_quiz_attempts_user_course ON quiz_attempts(user_id, course_id);
CREATE INDEX idx_quiz_attempts_passed ON quiz_attempts(user_id, course_id, passed);

-- =====
-- GROUP 7: Q&A
-- =====

```

```

CREATE TABLE qa_questions (
  id SERIAL PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  course_id INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  lesson_id INTEGER NOT NULL REFERENCES lessons(id) ON DELETE CASCADE,
  question_text TEXT NOT NULL,
  is_answered BOOLEAN NOT NULL DEFAULT FALSE,
  is_deleted BOOLEAN NOT NULL DEFAULT FALSE, -- admin soft delete
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

COMMENT ON TABLE qa_questions IS 'Student questions and feedback on specific lessons';
COMMENT ON COLUMN qa_questions.is_answered IS 'Set TRUE when admin submits a reply';
COMMENT ON COLUMN qa_questions.is_deleted IS 'Admin soft delete for spam or inappropriate questions';

```

```

CREATE INDEX idx_qa_questions_lesson ON qa_questions(lesson_id);
CREATE INDEX idx_qa_questions_course ON qa_questions(course_id);
CREATE INDEX idx_qa_questions_unanswered ON qa_questions(is_answered, is_deleted);

```

```

CREATE TABLE qa_replies (
  id SERIAL PRIMARY KEY,
  question_id INTEGER NOT NULL REFERENCES qa_questions(id) ON DELETE CASCADE,
  admin_id UUID NOT NULL REFERENCES profiles(id),
  reply_text TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

COMMENT ON TABLE qa_replies IS 'Admin replies to student Q&A questions. One reply per question';
COMMENT ON COLUMN qa_replies.admin_id IS 'Profile ID of the admin who replied';

```

```

CREATE INDEX idx_qa_replies_question ON qa_replies(question_id);

```

```

-- =====
-- GROUP 8: TRANSACTIONS
-- =====

```

```

CREATE TABLE payment_logs (
  id SERIAL PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  course_id INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,

```



```

razorpay_order_id    TEXT UNIQUE,           -- from Razorpay create order API
razorpay_payment_id  TEXT,                 -- from Razorpay webhook on success
razorpay_signature   TEXT,                 -- HMAC signature for verification
amount               INTEGER NOT NULL,      -- in INR (e.g. 100)
currency             TEXT NOT NULL DEFAULT 'INR',
status               TEXT NOT NULL DEFAULT 'created'
                    CHECK (status IN ('created', 'paid', 'failed')),
webhook_received_at  TIMESTAMPTZ,          -- when Razorpay webhook was received
created_at           TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at           TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

COMMENT ON TABLE payment_logs IS 'Razorpay payment records. Fully automatic – no manual entry';
COMMENT ON COLUMN payment_logs.razorpay_order_id IS 'Created by /api/payment/create-order';
COMMENT ON COLUMN payment_logs.razorpay_payment_id IS 'Set by webhook on successful payment';
COMMENT ON COLUMN payment_logs.status IS 'created=order initiated, paid=webhook confirmed, failed=payment failed';

```

```

CREATE INDEX idx_payment_user ON payment_logs(user_id);
CREATE INDEX idx_payment_status ON payment_logs(status);
CREATE INDEX idx_payment_order ON payment_logs(razorpay_order_id);

```

```

-- =====
-- GROUP 9: COMMUNITY
-- =====

```

```

CREATE TABLE testimonials (
  id          SERIAL PRIMARY KEY,
  user_id     UUID REFERENCES profiles(id) ON DELETE SET NULL,
  course_id   INTEGER REFERENCES courses(id) ON DELETE SET NULL,
  reviewer_name TEXT NOT NULL,
  content_en  TEXT NOT NULL,
  content_te  TEXT,
  rating      INTEGER NOT NULL DEFAULT 5 CHECK (rating BETWEEN 1 AND 5),
  is_published BOOLEAN NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

COMMENT ON TABLE testimonials IS 'Student reviews shown on home page';
COMMENT ON COLUMN testimonials.reviewer_name IS 'Stored separately – survives user deletion';

```

```

CREATE INDEX idx_testimonials_published ON testimonials(is_published);
CREATE INDEX idx_testimonials_course ON testimonials(course_id);

```

3. Automatic Timestamps

sql

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_paths_updated_at      BEFORE UPDATE ON paths      FOR EACH ROW
CREATE TRIGGER trg_courses_updated_at    BEFORE UPDATE ON courses    FOR EACH ROW
CREATE TRIGGER trg_lessons_updated_at    BEFORE UPDATE ON lessons    FOR EACH ROW
CREATE TRIGGER trg_profiles_updated_at   BEFORE UPDATE ON profiles   FOR EACH ROW
CREATE TRIGGER trg_user_notes_updated_at BEFORE UPDATE ON user_notes  FOR EACH ROW
CREATE TRIGGER trg_payment_logs_updated_at BEFORE UPDATE ON payment_logs  FOR EACH ROW
CREATE TRIGGER trg_watch_sessions_updated_at BEFORE UPDATE ON watch_sessions FOR EACH ROW
CREATE TRIGGER trg_qa_replies_updated_at BEFORE UPDATE ON qa_replies  FOR EACH ROW
```



4. Auto-create Profile on Registration

sql

```
CREATE OR REPLACE FUNCTION handle_new_user()
RETURNS TRIGGER AS $$
DECLARE
    fname TEXT;
    initials TEXT;
BEGIN
    fname := COALESCE(NEW.raw_user_meta_data->>'full_name', 'New User');
    initials := UPPER(
        LEFT(fname, 1) ||
        COALESCE(LEFT(SPLIT_PART(fname, ' ', 2), 1), '')
    );
    INSERT INTO profiles (id, full_name, avatar_initials)
    VALUES (NEW.id, fname, initials);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER trg_on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION handle_new_user();
```

5. Auto-activate Enrollment on Free/Intent Choices

When a student picks Option 2 or Option 3, activate enrollment immediately:

```
sql
```

```

CREATE OR REPLACE FUNCTION activate_free_enrollment()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.intent IN ('cannot_pay', 'not_willing_to_pay') THEN
        INSERT INTO enrollments (user_id, course_id, is_active, activated_at)
        VALUES (NEW.user_id, NEW.course_id, TRUE, NOW())
        ON CONFLICT (user_id, course_id)
        DO UPDATE SET is_active = TRUE, activated_at = NOW();
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_activate_free_enrollment
AFTER INSERT ON enrollment_intents
FOR EACH ROW EXECUTE FUNCTION activate_free_enrollment();

```

6. Auto-activate Enrollment on Razorpay Webhook

When Razorpay webhook updates payment status to **paid**:

```

sql

CREATE OR REPLACE FUNCTION activate_paid_enrollment()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'paid' AND OLD.status != 'paid' THEN
        INSERT INTO enrollments (user_id, course_id, is_active, activated_at)
        VALUES (NEW.user_id, NEW.course_id, TRUE, NOW())
        ON CONFLICT (user_id, course_id)
        DO UPDATE SET is_active = TRUE, activated_at = NOW();
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_activate_paid_enrollment
AFTER UPDATE ON payment_logs
FOR EACH ROW EXECUTE FUNCTION activate_paid_enrollment();

```

7. Auto-mark Q&A as Answered on Reply

sql

```
CREATE OR REPLACE FUNCTION mark_question_answered()  
RETURNS TRIGGER AS $$  
BEGIN  
    UPDATE qa_questions SET is_answered = TRUE WHERE id = NEW.question_id;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_mark_answered  
AFTER INSERT ON qa_replies  
FOR EACH ROW EXECUTE FUNCTION mark_question_answered();
```

8. Row Level Security (RLS)

sql

```

-- Enable RLS on all tables
ALTER TABLE paths          ENABLE ROW LEVEL SECURITY;
ALTER TABLE courses        ENABLE ROW LEVEL SECURITY;
ALTER TABLE lessons        ENABLE ROW LEVEL SECURITY;
ALTER TABLE quiz_questions ENABLE ROW LEVEL SECURITY;
ALTER TABLE profiles        ENABLE ROW LEVEL SECURITY;
ALTER TABLE enrollments     ENABLE ROW LEVEL SECURITY;
ALTER TABLE enrollment_intents ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_progress   ENABLE ROW LEVEL SECURITY;
ALTER TABLE watch_sessions  ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_notes      ENABLE ROW LEVEL SECURITY;
ALTER TABLE quiz_attempts   ENABLE ROW LEVEL SECURITY;
ALTER TABLE qa_questions    ENABLE ROW LEVEL SECURITY;
ALTER TABLE qa_replies      ENABLE ROW LEVEL SECURITY;
ALTER TABLE payment_logs    ENABLE ROW LEVEL SECURITY;
ALTER TABLE testimonials    ENABLE ROW LEVEL SECURITY;

-- Helper function
CREATE OR REPLACE FUNCTION is_admin()
RETURNS BOOLEAN AS $$
    SELECT EXISTS (
        SELECT 1 FROM profiles WHERE id = auth.uid() AND is_admin = TRUE
    );
$$ LANGUAGE sql SECURITY DEFINER;

-- PATHS
CREATE POLICY "paths_public_read" ON paths FOR SELECT USING (is_active = TRUE);
CREATE POLICY "paths_admin_all"   ON paths FOR ALL     USING (is_admin());

-- COURSES
CREATE POLICY "courses_public_read" ON courses FOR SELECT USING (is_published = TRUE);
CREATE POLICY "courses_admin_all"   ON courses FOR ALL     USING (is_admin());

-- LESSONS (youtube_video_id never returned to client via RLS – handled in API layer)
CREATE POLICY "lessons_enrolled_read" ON lessons FOR SELECT USING (
    is_preview = TRUE OR
    EXISTS (
        SELECT 1 FROM enrollments
        WHERE user_id = auth.uid() AND course_id = lessons.course_id AND is_active = TRUE
    )
);
CREATE POLICY "lessons_admin_all" ON lessons FOR ALL USING (is_admin());

```

-- QUIZ QUESTIONS

```
CREATE POLICY "quiz_enrolled_read" ON quiz_questions FOR SELECT USING (
    EXISTS (
        SELECT 1 FROM enrollments
        WHERE user_id = auth.uid() AND course_id = quiz_questions.course_id AND is_active
    )
);
CREATE POLICY "quiz_admin_all" ON quiz_questions FOR ALL USING (is_admin());
```

-- PROFILES

```
CREATE POLICY "profiles_own_read" ON profiles FOR SELECT USING (id = auth.uid());
CREATE POLICY "profiles_own_update" ON profiles FOR UPDATE USING (id = auth.uid());
CREATE POLICY "profiles_admin_read" ON profiles FOR SELECT USING (is_admin());
```

-- ENROLLMENTS

```
CREATE POLICY "enrollments_own_read" ON enrollments FOR SELECT USING (user_id = auth.uid());
CREATE POLICY "enrollments_own_insert" ON enrollments FOR INSERT WITH CHECK (user_id = auth.uid());
CREATE POLICY "enrollments_admin_all" ON enrollments FOR ALL USING (is_admin());
```

-- ENROLLMENT INTENTS

```
CREATE POLICY "intents_own_read" ON enrollment_intents FOR SELECT USING (user_id = auth.uid());
CREATE POLICY "intents_own_insert" ON enrollment_intents FOR INSERT WITH CHECK (user_id = auth.uid());
CREATE POLICY "intents_admin_read" ON enrollment_intents FOR SELECT USING (is_admin());
```

-- USER PROGRESS

```
CREATE POLICY "progress_own_read" ON user_progress FOR SELECT USING (user_id = auth.uid());
CREATE POLICY "progress_own_insert" ON user_progress FOR INSERT WITH CHECK (user_id = auth.uid());
CREATE POLICY "progress_own_delete" ON user_progress FOR DELETE USING (user_id = auth.uid());
CREATE POLICY "progress_admin_read" ON user_progress FOR SELECT USING (is_admin());
```

-- WATCH SESSIONS

```
CREATE POLICY "watch_own_read" ON watch_sessions FOR SELECT USING (user_id = auth.uid());
CREATE POLICY "watch_own_insert" ON watch_sessions FOR INSERT WITH CHECK (user_id = auth.uid());
CREATE POLICY "watch_own_update" ON watch_sessions FOR UPDATE USING (user_id = auth.uid());
CREATE POLICY "watch_admin_read" ON watch_sessions FOR SELECT USING (is_admin());
```

-- USER NOTES (private – admin cannot read)

```
CREATE POLICY "notes_own_read" ON user_notes FOR SELECT USING (user_id = auth.uid());
CREATE POLICY "notes_own_insert" ON user_notes FOR INSERT WITH CHECK (user_id = auth.uid());
CREATE POLICY "notes_own_update" ON user_notes FOR UPDATE USING (user_id = auth.uid());
```

-- QUIZ ATTEMPTS

```
CREATE POLICY "quiz_attempts_own_read" ON quiz_attempts FOR SELECT USING (user_id = auth.uid());
```

```

CREATE POLICY "quiz_attempts_own_insert" ON quiz_attempts FOR INSERT WITH CHECK (
CREATE POLICY "quiz_attempts_admin_read" ON quiz_attempts FOR SELECT USING (is_admin() = TRUE);

-- Q&A QUESTIONS (public to all enrolled students)
CREATE POLICY "qa_questions_enrolled_read" ON qa_questions FOR SELECT USING (
    is_deleted = FALSE AND
    EXISTS (
        SELECT 1 FROM enrollments
        WHERE user_id = auth.uid() AND course_id = qa_questions.course_id AND is_active = TRUE
    )
);
CREATE POLICY "qa_questions_own_insert" ON qa_questions FOR INSERT WITH CHECK (
    auth.uid() IS NOT NULL
);
CREATE POLICY "qa_questions_admin_all" ON qa_questions FOR ALL USING (is_admin());

-- Q&A REPLIES (public to all enrolled students)
CREATE POLICY "qa_replies_enrolled_read" ON qa_replies FOR SELECT USING (
    EXISTS (
        SELECT 1 FROM qa_questions q
        JOIN enrollments e ON e.course_id = q.course_id
        WHERE q.id = qa_replies.question_id
            AND e.user_id = auth.uid()
            AND e.is_active = TRUE
    )
);
CREATE POLICY "qa_replies_admin_all" ON qa_replies FOR ALL USING (is_admin());

-- PAYMENT LOGS
CREATE POLICY "payment_own_read" ON payment_logs FOR SELECT USING (user_id = auth.uid());
CREATE POLICY "payment_own_insert" ON payment_logs FOR INSERT WITH CHECK (user_id = auth.uid());
CREATE POLICY "payment_admin_all" ON payment_logs FOR ALL USING (is_admin());

-- TESTIMONIALS
CREATE POLICY "testimonials_public_read" ON testimonials FOR SELECT USING (is_public() = TRUE);
CREATE POLICY "testimonials_own_insert" ON testimonials FOR INSERT WITH CHECK (user_id = auth.uid());
CREATE POLICY "testimonials_admin_all" ON testimonials FOR ALL USING (is_admin());

```

9. Useful Views

9.1 Course Progress Summary

sql

```
CREATE OR REPLACE VIEW vw_course_progress AS
SELECT
    up.user_id,
    up.course_id,
    COUNT(up.lesson_id)           AS completed_lessons,
    total.total_lessons,
    ROUND(
        COUNT(up.lesson_id)::NUMERIC / NULLIF(total.total_lessons, 0) * 100
    )                             AS progress_pct
FROM user_progress up
JOIN (
    SELECT course_id, COUNT(*) AS total_lessons FROM lessons GROUP BY course_id
) total ON total.course_id = up.course_id
GROUP BY up.user_id, up.course_id, total.total_lessons;
```

9.2 My Courses (with progress)

sql

```
CREATE OR REPLACE VIEW vw_my_courses AS
SELECT
    e.user_id,
    e.enrolled_at,
    e.activated_at,
    c.id           AS course_id,
    c.slug,
    c.title_en,
    c.title_te,
    c.emoji,
    c.bg_color,
    c.instructor_en,
    c.instructor_te,
    c.duration,
    c.is_free,
    c.price,
    c.has_quiz,
    ei.intent,
    COALESCE(cp.progress_pct, 0)      AS progress_pct,
    COALESCE(cp.completed_lessons, 0) AS completed_lessons,
    COALESCE(cp.total_lessons, 0)     AS total_lessons
FROM enrollments e
JOIN courses c ON c.id = e.course_id
LEFT JOIN vw_course_progress cp ON cp.user_id = e.user_id AND cp.course_id = e.course_id
LEFT JOIN enrollment_intents ei ON ei.user_id = e.user_id AND ei.course_id = e.course_id
WHERE e.is_active = TRUE AND c.is_deleted = FALSE;
```

9.3 Enrollment Intent Analytics

sql

```

CREATE OR REPLACE VIEW vw_intent_analytics AS
SELECT
    DATE(ei.created_at)                AS date,
    c.title_en                        AS course,
    COUNT(*) FILTER (WHERE ei.intent = 'paid')        AS paid_count,
    COUNT(*) FILTER (WHERE ei.intent = 'cannot_pay')  AS cannot_pay_count,
    COUNT(*) FILTER (WHERE ei.intent = 'not_willing_to_pay') AS not_willing_count,
    SUM(ei.amount_paid)                AS total_revenue
FROM enrollment_intents ei
JOIN courses c ON c.id = ei.course_id
GROUP BY DATE(ei.created_at), c.title_en
ORDER BY date DESC;

```

9.4 Admin Q&A Hub View

```

sql

CREATE OR REPLACE VIEW vw_admin_qa AS
SELECT
    q.id                AS question_id,
    q.question_text,
    q.is_answered,
    q.is_deleted,
    q.created_at        AS asked_at,
    p.full_name         AS student_name,
    p.avatar_initials,
    c.title_en          AS course_title,
    c.slug              AS course_slug,
    l.title_en          AS lesson_title,
    l.order_index       AS lesson_order,
    r.reply_text,
    r.created_at        AS replied_at
FROM qa_questions q
JOIN profiles p  ON p.id  = q.user_id
JOIN courses c   ON c.id  = q.course_id
JOIN lessons l   ON l.id  = q.lesson_id
LEFT JOIN qa_replies r ON r.question_id = q.id
WHERE q.is_deleted = FALSE
ORDER BY q.created_at DESC;

```

9.5 Watch Analytics

sql

```
CREATE OR REPLACE VIEW vw_watch_analytics AS
SELECT
  ws.lesson_id,
  l.title_en          AS lesson_title,
  c.title_en          AS course_title,
  COUNT(DISTINCT ws.user_id)      AS unique_viewers,
  ROUND(AVG(ws.max_watch_pct))    AS avg_watch_pct,
  ROUND(AVG(ws.total_watch_seconds) / 60.0, 1) AS avg_watch_minutes
FROM watch_sessions ws
JOIN lessons l ON l.id = ws.lesson_id
JOIN courses c ON c.id = ws.course_id
GROUP BY ws.lesson_id, l.title_en, c.title_en
ORDER BY avg_watch_pct ASC;
```

10. Seed Data

10.1 Learning Paths

sql

```
INSERT INTO paths (slug, name, emoji, tagline_en, tagline_te, description_en, description_te, is_featured)
VALUES
  ('raas', 'RAAS', '🧘', 'Vedic & Spiritual Learning', 'వేద మరియు ఆధ్యాత్మిక అభ్యాసం',
  'A curated journey through the timeless wisdom of Vedic tradition.',
  'భగవద్గీత, ఉపనిషత్తులు, మంత్రాలు, యోగ ద్వారా వేద సంప్రదాయం యొక్క శాశ్వత జ్ఞానం.',
  TRUE, 1),
  ('daas', 'DAAS', '🌟', 'Coming Soon', 'త్వరలో వస్తోంది',
  'A new spiritual learning path – coming soon.',
  'ఒక కొత్త ఆధ్యాత్మిక అభ్యాస మార్గం – త్వరలో వస్తోంది.',
  FALSE, 2);
```

10.2 Courses (all RAAS, path_id = 1)

sql

```
INSERT INTO courses (path_id, slug, emoji, bg_color, title_en, title_te, description)
VALUES
```

```
(1, 'bhagavad-gita-essentials', '📖', '#1a0a00', 'Bhagavad Gita Essentials', 'భగవద్గీత టెస్టెంట్', 'భగవద్గీత టెస్టెంట్'),
(1, 'vedic-mantra-chanting', 'ॐ', '#001a0a', 'Vedic Mantra Chanting', 'వేద మంత్ర పఠనం', 'వేద మంత్ర పఠనం'),
(1, 'upanishads-inner-light', '💡', '#150010', 'Upanishads: Inner Light', 'ఉపనిషత్తులు: అంతర్గత ప్రకాశం', 'ఉపనిషత్తులు: అంతర్గత ప్రకాశం'),
(1, 'puja-vidhi-and-rituals', '🌸', '#001a0a', 'Puja Vidhi and Rituals', 'పూజా విధి మరియు రీతులు', 'పూజా విధి మరియు రీతులు'),
(1, 'yoga-and-vedic-philosophy', '🧘', '#0a001a', 'Yoga and Vedic Philosophy', 'యోగ మరియు వేద ఫిలోసఫీ', 'యోగ మరియు వేద ఫిలోసఫీ'),
(1, 'ramayana-for-beginners', '📖', '#1a0a00', 'Ramayana for Beginners', 'రామాయణం - ప్రారంభింప', 'రామాయణం - ప్రారంభింప'),
(1, 'shiva-purana-stories', '🕉️', '#1a0005', 'Shiva Purana Stories', 'శివ పురాణ కథలు', 'శివ పురాణ కథలు'),
(1, 'devi-mahatmyam', '🌺', '#001a10', 'Devi Mahatmyam', 'దేవి మాహాత్మ్యం', '700 verses glorifying Goddess', 'దేవి మాహాత్మ్యం, 700 పద్యాల గౌరవనీయత'),
(1, 'carnatic-devotional-music', '🎵', '#0a1a00', 'Carnatic Devotional Music', 'కర్ణాటక భజన గీతలు', 'కర్ణాటక భజన గీతలు'),
(1, 'sanskrit-for-beginners', '📖', '#1a0a00', 'Sanskrit for Beginners', 'సంస్కృతం - ప్రారంభింప', 'సంస్కృతం - ప్రారంభింప'),
(1, 'jyotisha-vedic-astrology', '🌙', '#0a001a', 'Jyotisha: Vedic Astrology', 'జ్యోతిష్యం - వేద జ్యోతిష్యం', 'జ్యోతిష్యం - వేద జ్యోతిష్యం'),
(1, 'vastu-shastra-basics', '🏠', '#001a15', 'Vastu Shastra Basics', 'వాస్తు శాస్త్రం - పరిచయం', 'వాస్తు శాస్త్రం - పరిచయం')
```

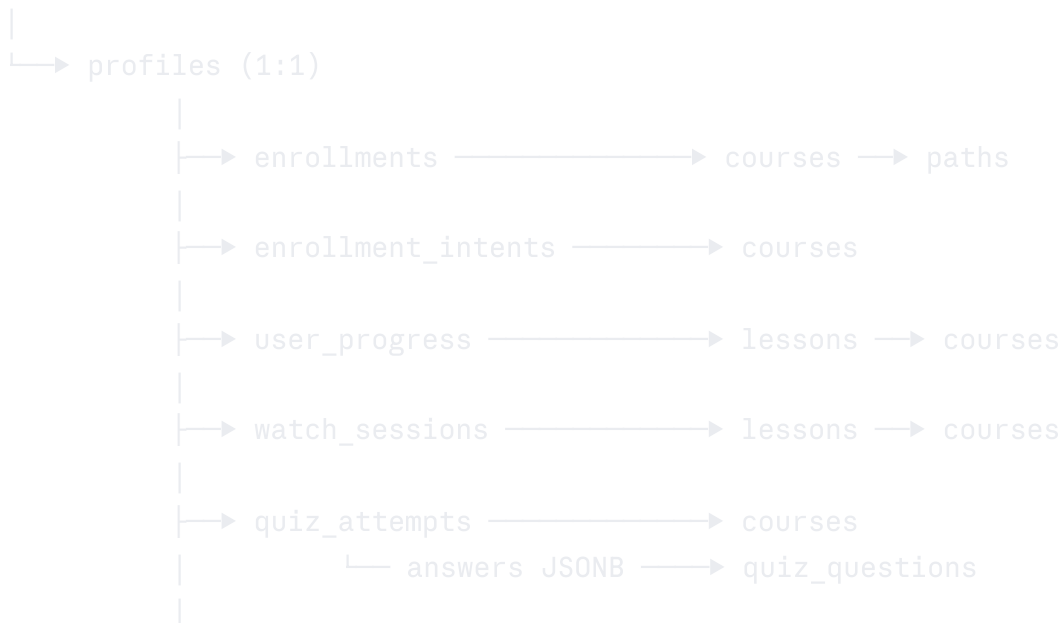
10.3 Set Admin Flag

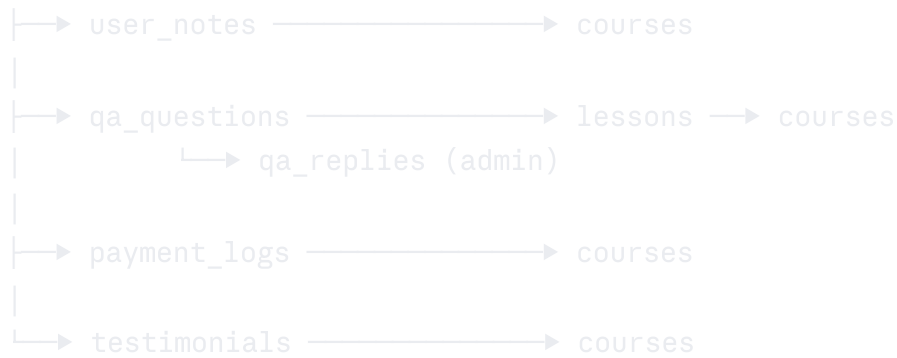
```
sql
```

```
UPDATE profiles
SET is_admin = TRUE
WHERE id = (SELECT id FROM auth.users WHERE email = 'admin@krishnamargam.in');
```

11. Key Relationships

```
auth.users (Supabase managed)
```





12. Design Decisions

Decision	Reason
<code>enrollment_intents</code> separate from <code>enrollments</code>	Clean separation of access control (<code>enrollments</code>) from analytics data (<code>enrollment_intents</code>)
Default price <code>100</code> on courses	Reflects ₹100 default. Admin can override. Seed data uses 0 for explicitly free courses
<code>youtube_video_id</code> in <code>lessons</code> only — never in API response	URL masking — video ID served via short-lived signed token from API, never in client HTML
<code>watch_sessions</code> per day	Avoids row explosion for active students; one row updated per lesson per day
<code>quiz_attempts</code> JSONB for answers	Flexible — any number of questions, answers stored as map without schema changes
Triggers for enrollment activation	Atomic — no race condition between intent recording and enrollment activation
<code>is_admin()</code> helper function in RLS	Avoids repeated subquery in every policy; cleaner and more maintainable
<code>user_notes</code> private (no admin RLS)	Student notes are personal — admin has no business reading them
Soft delete on <code>courses</code> and <code>qa_questions</code>	Preserves history and enrolled student data
<code>reviewer_name</code> denormalised on testimonials	Testimonials survive user account deletion

